UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

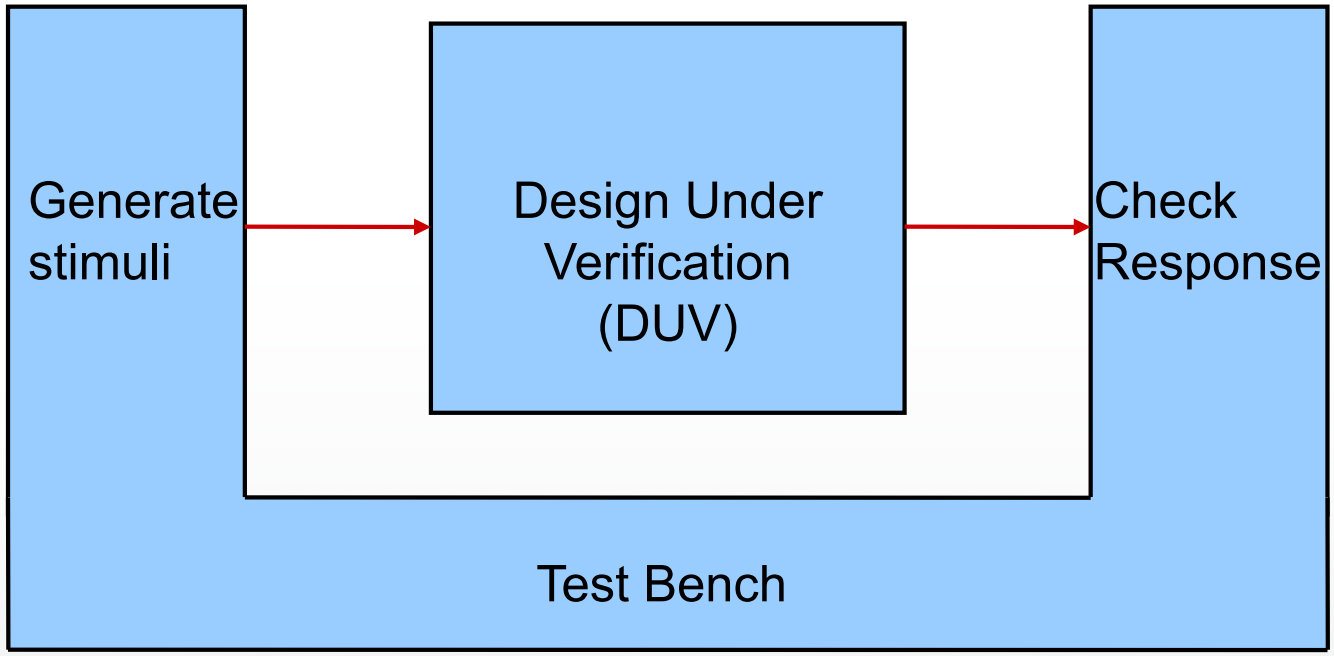
# 28: Verification

UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

# 28.1 Testbench Strategies

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# The Basic Testbench



UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Verification Plan

- As the design develops, do you want to keep testing the same things?
  - Check to see that the design has not been broken (regression testing)
  - Check new things
- One simulation run? Or a number of smaller simulations?
- Need to write a verification plan – what to verify, when, how

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Divide and Conquer

- Your design isn't one huge monolithic block
- Why should the testbench be like that?
- Issues:
  - Verification plan probably requires many different tests
  - Need to test several versions of the design
  - Create environment around the design
- Objectives:
  - Flexibility
  - Low-effort maintenance and enhancement
  - Reliability
  - Re-use

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Monolithic Testbenches are Inflexible

- Consider what would need to be changed to run different test cases
  - *test case* = group of tests required by one aspect of the verification plan, run as one simulation
- Stimulus generator and file output writers would need to change
- Top-level testbench then changes to accommodate them

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Package Useful Functionality

- High-level component models for other parts of the system
- Stimulus generator
- Output checker
- Logging and monitoring
- Utility functions (application-specific calculations)

UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

# 28.2 Verification Methodologies

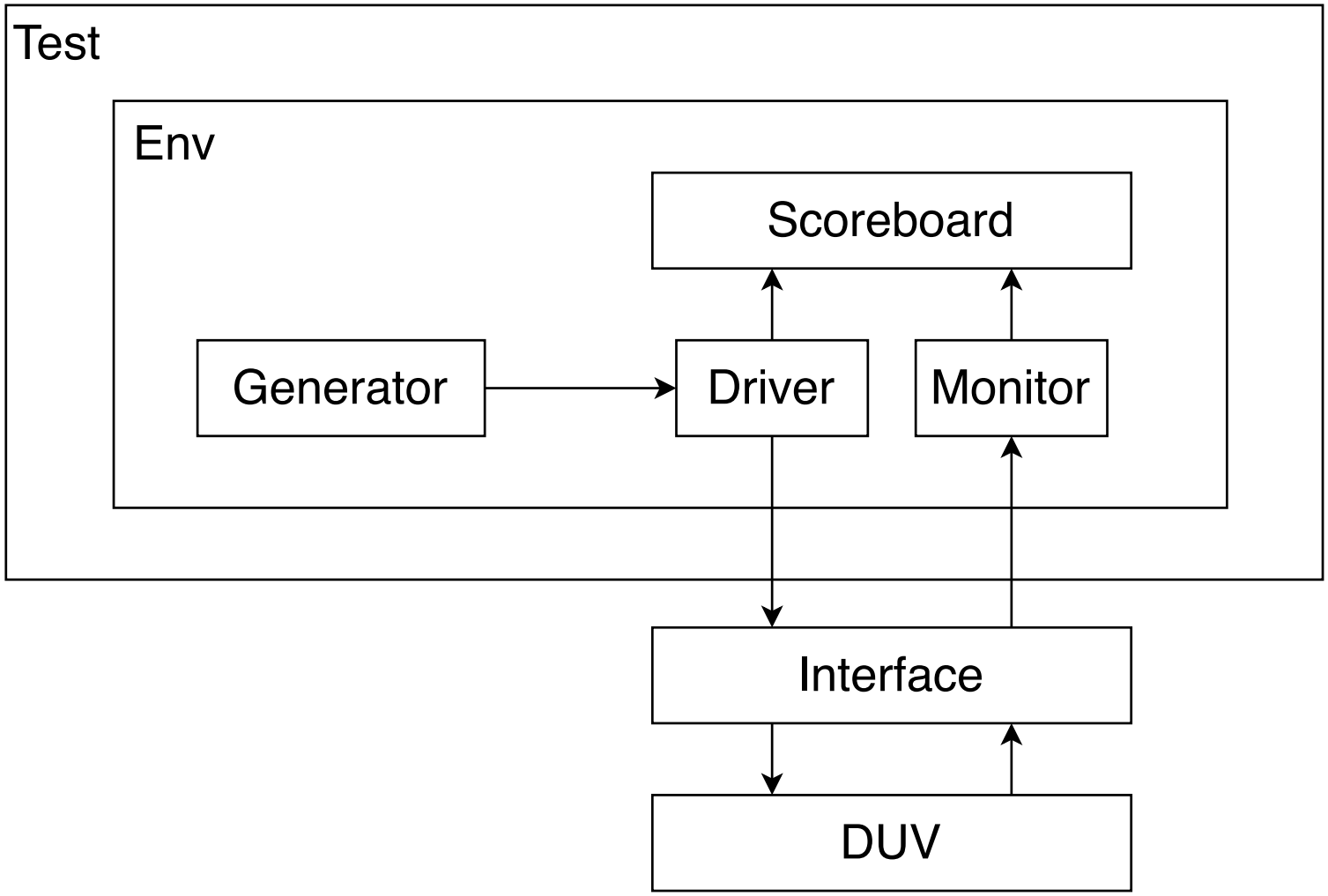
UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Standards

- Universal Verification Methodology (UVM) is an industry standard for verifying designs.
  - SystemVerilog class library for simulation
- cocotb is an open source approach backed by Synopsys, Cadence, Siemens, Aldec
  - Python class library
- Similar objectives, but very complex
- Here, we will look at a simplified approach

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Testbench structure



UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

# 28.3 Coverage

UNIVERSITY OF Southampton
School of Electronics and Computer Science

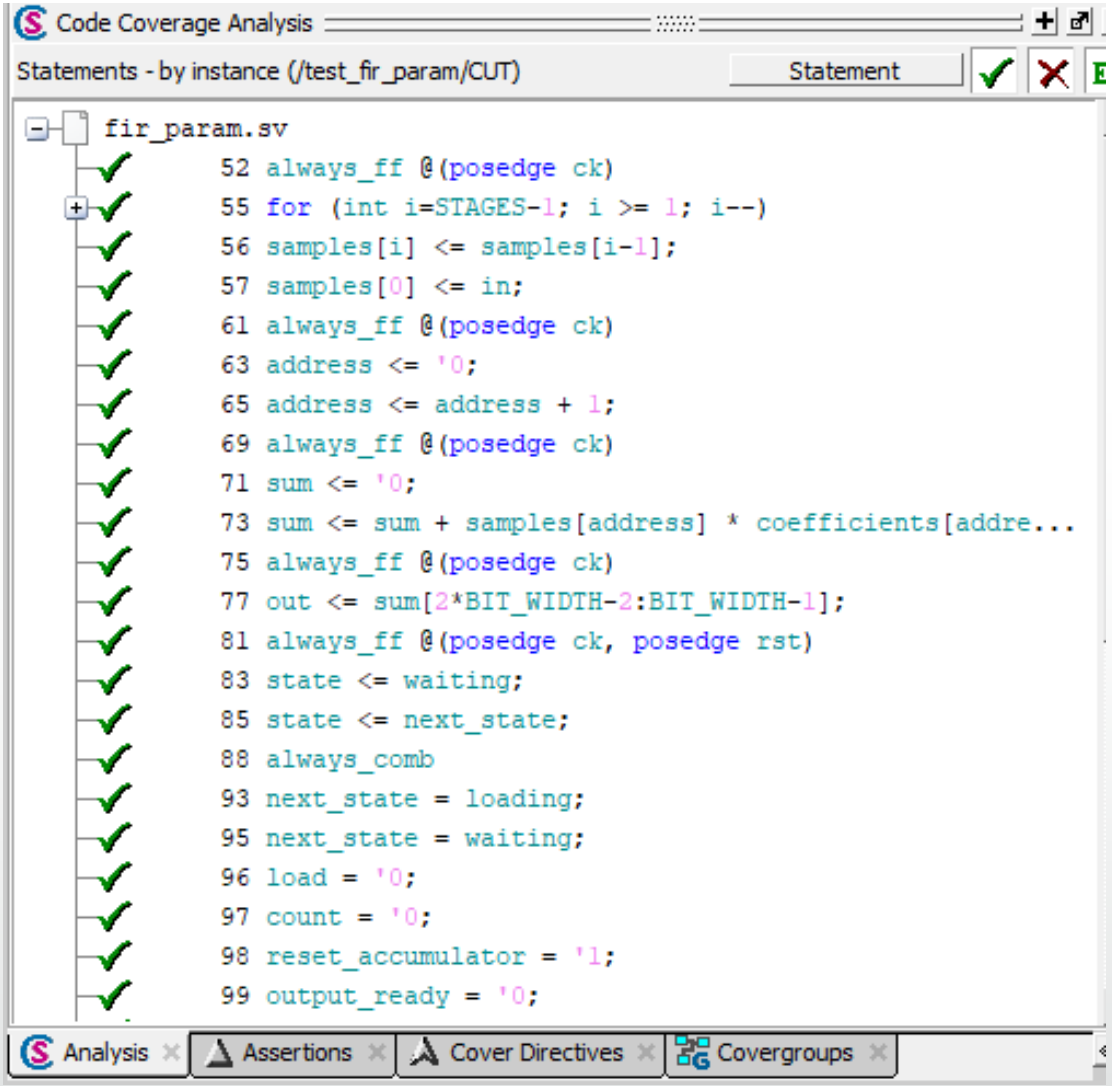
# Coverage

- The quality of a test can be defined in terms of its *coverage*
- Various metrics:
  - Lines of code
  - Branches in code
  - Range of outputs
  - Bits toggled (0 -> 1, 1 ->0)
  - Assertions

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Statement Coverage

- After simulating for some cycles, we can see that every line has been executed
- Good! But what does this tell us about meeting the specification?
- Same with branches, toggle
- AI/ML tools can generate tests for this kind of cover



UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Example

```
always_ff @(posedge clk, negedge n_reset)
    q <= d
```

- Code coverage would tell us that the line has been exercised
  - It won't tell us that we're forgotten to include the reset logic
- On the other hand, if the behaviour is not correct, we might see that one or more lines are never executed

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Functional Coverage

- SystemVerilog includes covergroups and coverpoints
- A `coverpoint` is something that is measured
- A `covergroup` is like a class and may contain several coverpoints
- The coverpoint collects the values that a signal or variable, etc. takes and puts these values into "bins"

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Example

- using the testbench from the last lecture

```
covergroup CovQ @(vif.ready);
   coverpoint vif.qout;
endgroup
```

- This is instatiated in the testbench initial block with:

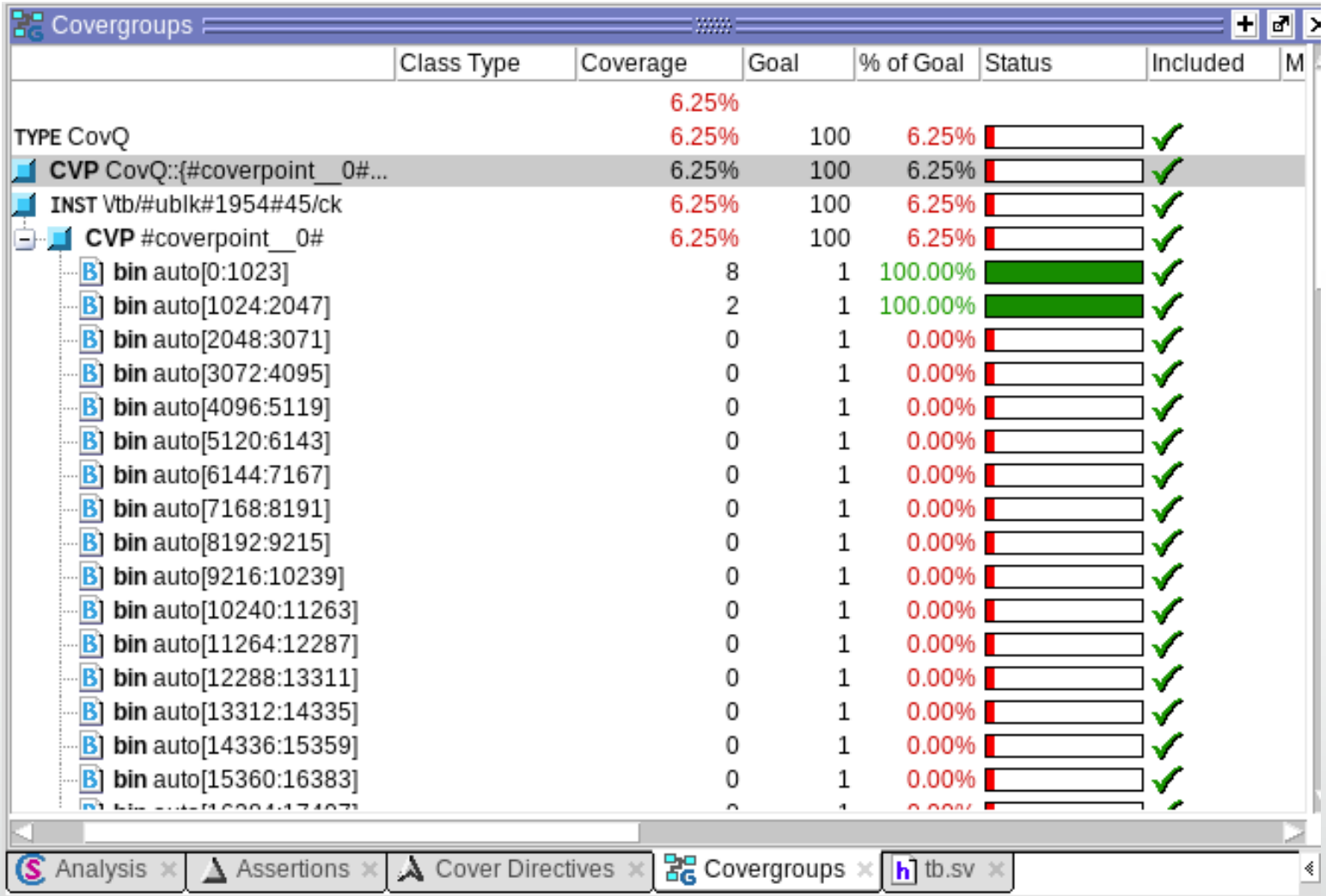
```
CovQ ck;
ck = new();
```

- Could do this in the scoreboard or elsewhere

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Coverage Report

- In the coverage window, we see something like this:



UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Cover bins

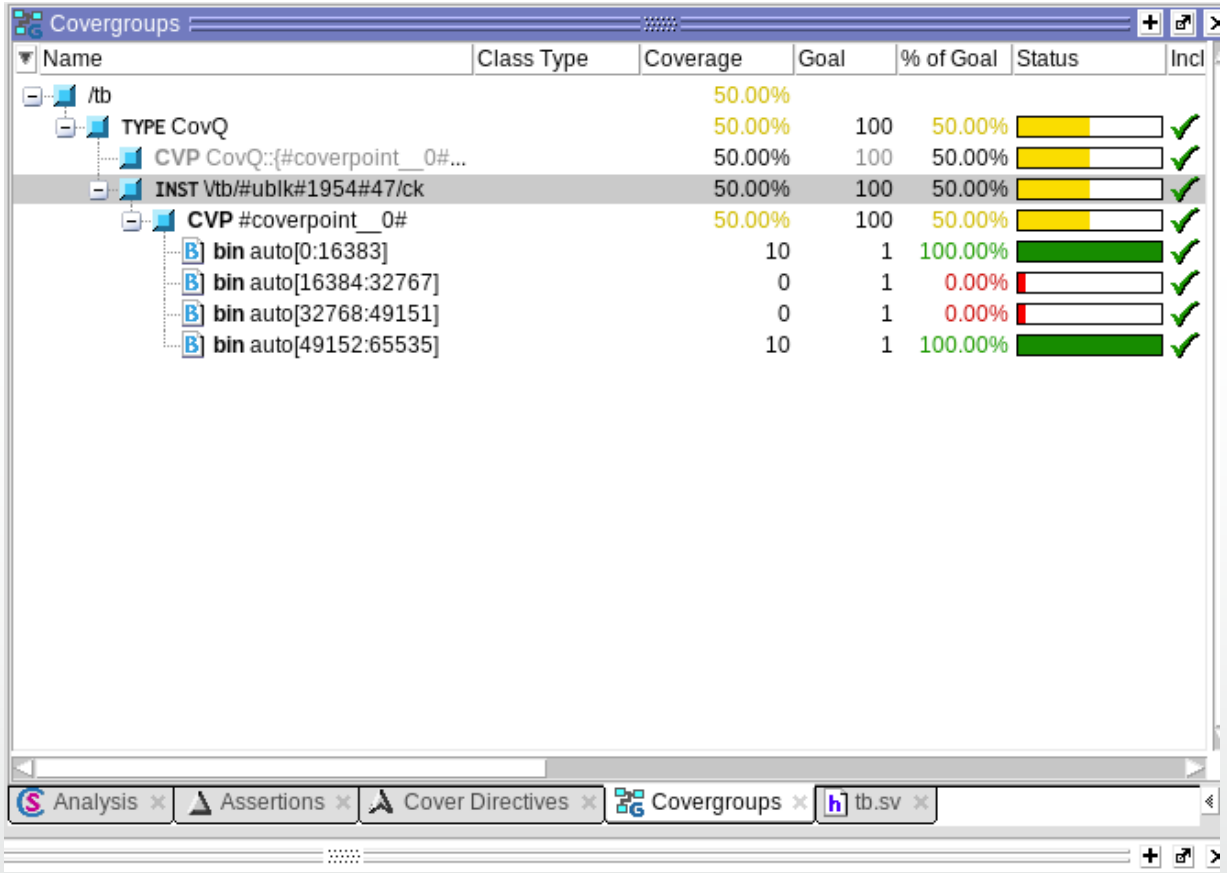
- This report doesn't tell us much
- By default, 64 equal size bins are created.
  - In this case, each bin covers 1024 output values from a multiplier
  - Only the first two bins and the last two bins (not shown) have collected any samples.
- We could limit the collection to, say, 4 bins:

```
coverpoint vif.qout
 {    option.auto_bin_max = 4;}
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Cover bins

- This tells us that two of the automatic ranges are covered, but two are not
- Arguably, this is not useful for a multiplier
- We don't care about values – we care about corner cases



UNIVERSITY OF Southampton
School of Electronics and Computer Science

# User defined bins

- We can define our own bins, and also warn against ignored and illegal values

```
covergroup CovKind;
coverpoint tr.kind { //kind has 4 bits, 16 values
    bins zero = {0}; // A bin for kind==0
    bins lo = {[1:3]}; // 1 bin for values 1:3
    ignore_bins ig = {4};
    illegal_bins ill = {5};
    bins hi[] = {[8:$]}; // 8 separate bins
    bins misc = default; // 1 bin for all the rest
}
endgroup // CoverKind
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Cross Cover

- Cross cover – what values were seen at two or more cover points at the same time?

```
class Transaction;
  rand bit [3:0] kind;
  rand bit [2:0] port;
endclass

Transaction tr;

covergroup CovPort;
  kind: coverpoint tr.kind; // Create cover point kind
  port: coverpoint tr.port // Create cover point port
  cross kind, port; // Cross kind and port
endgroup
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Cross Cover Report

```
Cumulative report for Transaction::CovPort
Summary:
    Coverage: 95.83
    Goal: 100

Coverpoint           Coverage     Goal         Weight
=======================================================
kind                   100.00      100         1
port                   100.00      100         1
===============|=======================================
Cross                  Coverage    Goal         Weight
=======================================================
Transaction::CovPort   87.50       100         1

Cross Coverage report
CoverageGroup: Transaction::CovPort
    Cross: Transaction::CovPort
Summary
    Coverage: 87.50
    Goal: 100
    Coverpoints Crossed: kind port
    Number of Expected Cross Bins: 128
    Number of User Defined Cross Bins: 0
    Number of Automatically Generated Cross Bins: 112

    Automatically Generated Cross Bins

    kind       port           # hits       at least
 ================================================
    auto[0]    auto[0]        1            1
    auto[0]    auto[1]        4            1
    auto[0]    auto[2]        3            1
    auto[0]    auto[5]        1            1
...
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Constrained Random Test

- Verification (Test) sequences can be *directed* or *random*
  - Cross cover example showed coverage with respect to random patterns
  - But some random patterns may not be interesting, or may be meaningless
  - For example, suppose a test bench generates random instructions for testing a CPU
  - Instructions that access memory should only have meaningful memory addresses
    - Or, perhaps, we are only interested in out-of-range addresses

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Constraints

- Constraints can be declared on random values

```
rand logic [7:0]     ain;
  constraint a {ain <20; }
```

- Multiple constraints can be declared
  - This is more complicated than it appears
  - The simulator has to use a satisfiability (SAT) solver to ensure that the constraints are consistent and achievable

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Constraint example

```
typedef enum {ADD, SUB, DIV, MOV} inst_e;

rand int m_int;
rand bit[3:0] m_array[];
rand inst_e inst;

// Randomization constraints.
constraint c {
  m_int > 500;
  m_int <= 1000;
  m_array.size() < 5;
  inst inside {ADD, DIV};
  if (m_int > 700) {
    m_array.size() == 1; }
}
```

UNIVERSITY OF Southampton
School of Electronics and Computer Science

# Verification Summary

- Verification is difficult!
  - SystemVerilog for design is easy
    - Well-defined objective – synthesise hardware
    - Well-established models
  - SystemVerilog (or Python) for verification is hard
    - What are the objectives?
    - Methodologies are complex, intended for large teams, large designs
    - Relatively new approaches
    - Open source tools (Verilator) struggle with some constructs